

DriveLegal BIMSTEC AI

Master Prompt Architecture & Project Blueprint

No-Docker Local Dev Edition

■■ Bangladesh ■■ Bhutan ■■ India ■■ Myanmar ■■ Nepal ■■ Sri Lanka ■■ Thailand

6 Countries

2,400+ Rules

12 Languages

Offline-First

No Docker

01 MASTER AI PROMPT ARCHITECTURE

System prompt template powering every AI response in the app

System Prompt (ragEngine.ts / gemini.service.ts)

You are DriveLegal AI – the authoritative traffic law assistant for BIMSTEC nations.

IDENTITY

- Expert on traffic laws: Bangladesh, Bhutan, India, Myanmar, Nepal, Sri Lanka, Thailand
- Always cite the specific Act / Section when giving legal info
- Never fabricate laws. If unsure, say: "Please verify with local traffic authority."

CONTEXT INJECTION (RAG)

{{RETRIEVED_LAW_CHUNKS}}	← top-k vector search results
{{USER_COUNTRY}}	← auto-detected or user-selected
{{USER_LANGUAGE}}	← for response language
{{GPS_CONTEXT}}	← border-crossing awareness

RESPONSE FORMAT

- Lead with the direct answer in one sentence
- Follow with legal citation (Act, Section, Penalty amount)
- Currency in local denomination + USD equivalent
- End with a safety tip when relevant

TOURIST MODE (if tourist=true)

- Explain in simple English first, then local language
- Flag left/right-drive transitions explicitly
- Highlight cross-border law changes proactively

VIOLATION MODE (if violation_detected=true)

- State violation clearly
- Show fine breakdown: base + surcharge + court fee
- Suggest payment options available in that country

EMERGENCY MODE (if sos=true)

- Respond ONLY in the user's language
- Provide exact emergency numbers for {{USER_COUNTRY}}
- Output structured FIR checklist immediately

Dynamic Context Builder — promptBuilder.ts

```
export function buildPrompt(ctx: PromptContext): string {
  return systemPrompt
    .replace('{{RETRIEVED_LAW_CHUNKS}}', ctx.ragChunks.join('\n--\n'))
    .replace('{{USER_COUNTRY}}', ctx.country)
    .replace('{{USER_LANGUAGE}}', ctx.language)
    .replace('{{GPS_CONTEXT}}', ctx.gpsContext ?? 'N/A');
}
```

RAG Pipeline — ragEngine.ts

```
async function query(userInput: string, country: string) {
  const embedding = await embedText(userInput);           // OpenAI / Gemini embed
  const chunks    = await pinecone.query({                // top-5 law chunks
    vector: embedding, topK: 5,
    filter: { country }
  });
  const prompt = buildPrompt({ ragChunks: chunks, country, language, gpsContext });
  return await gemini.generateContent(prompt);
}
```

02 FINAL PROJECT FOLDER STRUCTURE

No docker — local dev only

Root Layout

```
drivelegal-bimstec-ai/
■■■■ frontend/      ← React + Vite + Tailwind
■■■■ backend/       ← Node.js + Express + TypeScript
■■■■ data/          ← Master law JSON + embeddings
■■■■ ml-models/     ← TFLite models (helmet, OCR, signs)
■■■■ docs/          ← API.md, ARCHITECTURE.md, DEMO-SCRIPT.md
■■■■ scripts/       ← seed-database.ts, generate-embeddings.ts
■■■■ README.md
■■■■ .gitignore     ← (no docker/ folder needed)
```

Frontend — src/

```
src/
■■■■ main.tsx        ← Vite entry
■■■■ App.tsx         ← Router root
■■■■ components/
■   ■■■■ ai/
■   ■   ■■■■ ChatInterface.tsx ← Main chat UI
■   ■   ■■■■ MessageBubble.tsx
■   ■   ■■■■ VoiceInput.tsx    ← Web Speech API
■   ■   ■■■■ SuggestedQuestions.tsx
■   ■■■■ country/
■   ■   ■■■■ CountrySelector.tsx
■   ■   ■■■■ CountryComparison.tsx
■   ■   ■■■■ BorderCrossingAlert.tsx
■   ■■■■ challan/
■   ■   ■■■■ ChallanCalculator.tsx
■   ■   ■■■■ ViolationSelector.tsx
■   ■   ■■■■ FineBreakdown.tsx
■   ■■■■ vision/
■   ■   ■■■■ CameraCapture.tsx
■   ■   ■■■■ SignRecognition.tsx
■   ■   ■■■■ HelmetDetection.tsx
```

```

■   ■■■ emergency/
■       ■■■ SOSButton.tsx
■       ■■■ AccidentChecklist.tsx
■■■ pages/
■   ■■■ Home.tsx           ← Landing
■   ■■■ ChatPage.tsx
■   ■■■ ChallanPage.tsx
■   ■■■ CountryDashboard.tsx
■   ■■■ TouristAssistant.tsx
■   ■■■ EmergencyHelp.tsx
■■■ services/
■   ■■■ api/               ← axios wrappers per feature
■   ■■■ ai/
■       ■   ■■■ ragEngine.ts    ← Client-side RAG helper
■       ■   ■■■ promptBuilder.ts
■       ■   ■■■ translator.ts
■   ■■■ offline/
■       ■■■ indexedDB.ts      ← Dexie.js offline store
■       ■■■ syncManager.ts
■■■ store/                 ← Zustand slices
■■■ data/                  ← Static JSON (laws, contacts, i18n)
■■■ hooks/                 ← useAI, useGeolocation, useVoice...

```

Backend — src/

```

src/
■■■ server.ts              ← Entry: PORT=5000
■■■ app.ts                 ← Express + middleware
■■■ routes/
■   ■■■ ai.routes.ts       ← POST /api/ai/chat
■   ■■■ challan.routes.ts  ← GET /api/challan/calculate
■   ■■■ country.routes.ts  ← GET /api/country/:id/laws
■   ■■■ auth.routes.ts
■   ■■■ emergency.routes.ts
■   ■■■ vision.routes.ts   ← POST /api/vision/analyze
■■■ services/
■   ■■■ ai/
■       ■   ■■■ gemini.service.ts ← Gemini Pro API calls
■       ■   ■■■ rag.service.ts    ← Pinecone vector search
■       ■   ■■■ translation.service.ts
■   ■■■ rules/
■       ■■■ ruleEngine.ts        ← Law matching logic
■       ■■■ challanCalculator.ts
■■■ models/                ← Prisma models (User, Chat, Challan..)
■■■ middleware/            ← auth, rateLimit, cors, error
■■■ config/
■   ■■■ database.ts          ← Neon/Supabase Postgres connection
■   ■■■ redis.ts             ← Upstash Redis
■   ■■■ ai.config.ts         ← Gemini + Pinecone keys
■■■ prisma/
■   ■■■ schema.prisma
■   ■■■ seed.ts

```

Data Layer — data/

```

data/
■■■ traffic-laws/
■   ■■■ nepal/
■       ■   ■■■ helmet-laws.json
■       ■   ■■■ speed-limits.json
■       ■   ■■■ dui-laws.json
■       ■   ■■■ penalties.json
■   ■■■ bangladesh/ ■■■ bhutan/ ■■■ thailand/
■   ■■■ myanmar/    ■■■ srilanka/
■■■ embeddings/
■   ■■■ law-embeddings.json ← pre-computed vectors (run once)
■■■ schemas/

```

```
■■■ country.schema.json
■■■ violation.schema.json
■■■ challan.schema.json
```

03 LOCAL DEV SETUP — NO DOCKER

Everything you need to run in two terminals

One-time Setup

```
# 1. Clone & install
git clone && cd drivelegal-bimstec-ai

# 2. Frontend
cd frontend && npm install

# 3. Backend
cd ../backend && npm install

# 4. Copy env files
cp frontend/.env.example frontend/.env
cp backend/.env.example backend/.env
```

Environment Variables — backend/.env

```
DATABASE_URL=postgresql://...    ← Neon or Supabase (free)
REDIS_URL=rediss://...           ← Upstash Redis (free)
GEMINI_API_KEY=...               ← Google AI Studio (free)
PINECONE_API_KEY=...             ← Pinecone free tier
PINECONE_INDEX=drivelegal-laws
JWT_SECRET=your_secret_here
PORT=5000
```

Run Everything

```
# Terminal 1 – Frontend (http://localhost:5173)
cd frontend && npm run dev

# Terminal 2 – Backend (http://localhost:5000)
cd backend && npm run dev

# One-time: seed DB + generate embeddings
cd backend && npx ts-node prisma/seed.ts
npx ts-node scripts/generate-embeddings.ts
```

Hosted Free Tiers replacing Docker services

Service	Local Docker was...	Replace With	Cost
PostgreSQL	postgres container	Neon.tech	Free
Redis	redis container	Upstash.com	Free
Vector DB	local Qdrant	Pinecone free tier	Free
LLM	Ollama local	Gemini Pro API	Free tier

04 KEY FEATURE PROMPT TEMPLATES

Tourist Mode Prompt

TOURIST MODE ACTIVATED – Country: {{country}}, Route: {{route}}

The user is a foreign tourist. Respond in English first, then {{native_lang}}.

Proactively flag:

1. Drive side change (left/right) at border crossings
2. Helmet requirements for passengers (varies by country)
3. Speed limit units (km/h vs mph)
4. Documents required at each border checkpoint

Current location context: {{gps_context}}

Upcoming border crossing: {{next_border}} in ~{{km_to_border}} km

Challan / Fine Calculation Prompt

CHALLAN CALCULATION REQUEST

Country: {{country}} | Violation: {{violation_type}} | Vehicle: {{vehicle_class}}

Retrieve the exact fine from the {{country}} Motor Vehicles Act.

Output JSON:

```
{
  "violation": "...",
  "base_fine": ,
  "surcharge": ,
  "court_fee": ,
  "total": ,
  "currency": "...",
  "usd_equivalent": ,
  "legal_section": "Section X of Act Y",
  "payment_methods": ["online", "challan_counter", ...]
}
```

Vision / Camera Analysis Prompt

VISION ANALYSIS – Image provided by user camera.

Tasks (run in order):

1. Detect traffic sign → explain in {{language}}
2. Detect helmet presence on riders → flag violation if absent
3. Read number plate if visible → return plate string
4. Identify violation type from scene → map to fine table

Country context: {{country}}

Response must include: detected_items[], violations[], confidence_scores[]

Emergency / SOS Prompt

SOS MODE – HIGHEST PRIORITY RESPONSE

User location: {{gps_lat}}, {{gps_long}} | Country: {{country}}

Respond ONLY in {{user_language}}. Output:

1. Emergency numbers for {{country}} (police, ambulance, fire)
2. GPS coordinates to share with emergency services
3. FIR (First Information Report) checklist for accident
4. Nearest hospital (if maps API available)
5. Insurance claim first steps

Do NOT include any non-emergency information.

05 5-MINUTE HACKATHON DEMO SCRIPT

ACT 1 — The Problem

30s

- Show BIMSTEC traffic footage or stat card

- "242,000 road deaths yearly. Tourists fined unfairly. Laws differ every border."
- Highlight the 6-country complexity visually

ACT 2A — Tourist Mode

45s

- Input: 'I am driving Kathmandu to Bangkok'
- AI shows route with law-change markers at each border
- Highlights: left-drive to right-drive transition at Myanmar
- Auto-translates Thai signs in camera view

ACT 2B — Live Violation Detection

45s

- Point camera at helmet-less rider photo
- AI: Violation detected — Rs.500 fine (Nepal Motor Vehicles Act S.146)
- Shows fine breakdown + payment QR code

ACT 2C — Voice Assistant

45s

- Hands-free: "What is the drunk driving limit here?"
- AI responds with voice + visual overlay showing BAC limits
- Demonstrates offline fallback with cached laws

ACT 2D — Emergency SOS

45s

- Tap SOS button — simulated accident scenario
- Auto-shares GPS + calls ambulance number
- Shows FIR checklist rendered in Bengali

ACT 3 — Impact & Scale

90s

- Live dashboard: 6 countries, 2,400+ rules, 12 languages
- Offline-first architecture slide
- Expansion roadmap: ASEAN, SAARC, EU
- Monetisation: Gov API licensing + B2B fleet management

06 TECH STACK SUMMARY

Layer	Technology	Purpose
Frontend	React + Vite + Tailwind	PWA, offline-first UI
State	Zustand	Lightweight global state
Backend	Node.js + Express + TS	REST API server
AI/LLM	Gemini Pro API	Chat, translation, analysis
RAG	Pinecone + embeddings	Law chunk retrieval
Database	PostgreSQL via Neon	Users, challans, logs
Cache	Redis via Upstash	Sessions, rate limits
Offline DB	IndexedDB (Dexie.js)	Client-side law cache
Vision	TensorFlow.js + Tesseract	Helmet detect, OCR, signs
Maps	Leaflet.js + OpenStreetMap	Routes, geofencing
Voice	Web Speech API	Hands-free input/output

Auth	JWT + bcrypt	Secure sessions
Deploy	Railway / Render	No Docker, push-to-deploy

DriveLegal BIMSTEC AI

Safer Roads. Smarter Laws. One AI.

Hackathon Build — No
Docker